

U.D. 8. PL-SQL. CONTINUACIÓN.....



BBDD

Nuria Fuentes Pérez

INDICE

- 5. CURSORES.
 - A. Implícitos.
 - B. Explícitos.
- 6. EXCEPCIONES.
- 7. Cursores de actualización.
- 8. TRIGGERS.
- 9. REGISTROS.
- 10. TABLAS (arrays).

5. CURSORES.

CURSORES

Un cursor sirve para guardar el resultado de una consulta.

El resultado de cualquier instrucción SQL va a un área de memoria a la que accede el cursor.

Una sentencia **SELECT** puede devolver cero, una o más de una fila y eso hace que distingamos 2 tipos de cursores:

- Cursores implícitos
sentencias que devuelven una sola fila
- Cursores explícitos
sentencias que devuelven más de una fila

5. CURSORES.

A. Cursores implícitos.

Consultas que devuelvan una sola fila guardando el resultado en variables

```
SELECT columna/s INTO variable/s FROM tabla/s;
```

```
DECLARE
dept DEPART.DEPT_NO%TYPE;
total NUMBER(10);
BEGIN
  SELECT DEPT_NO, SUM(SALARIO) INTO dept, total
    FROM EMPLE
   WHERE DEPT_NO=20
   GROUP BY DEPT_NO;
  DBMS_OUTPUT.PUT_LINE(dept||': '||total);
END;
```

5. CURSORES.

A. Cursores implícitos.

El valor de los atributos del cursor implícito se refiere a la última orden SQL ejecutada (aunque el cursor ya esté cerrado):

- **SQL%FOUND.** → Booleano. Devuelve *true* si el último INSERT, UPDATE, DELETE o SELECT INTO han afectado a algún valor, en otro caso devuelve *false*.
- **SQL%NOTFOUND.** → Booleano. Devuelve *true* si el último INSERT, UPDATE, DELETE o SELECT INTO no han afectado a ninguna fila, en otro caso devuelve *false*.
- **SQL%ROWCOUNT.** → número entero. Devuelve el número de filas afectadas por el último INSERT, UPDATE, DELETE o SELECT INTO.
- **SQL%ISOPEN** → Booleano. Siempre devolverá *false* ya que Oracle cierra automáticamente el cursor después de cada orden SQL.

5. CURSORES.

A. Cursores implícitos.

✓ SELECT INTO debe devolver una y sólo una fila si no es así se lanzará una excepción:

- NO_DATA_FOUND → si no devuelve ninguna fila
- TOO_MANY_ROWS → si devuelve más de una

✓ INSERT, UPDATE, DELETE no se lanzan éstas excepciones

```
DECLARE
  nomDepto DEPART.DNOMBRE%TYPE;
  locDepto DEPART.LOC%TYPE;
BEGIN
  nomDepto:='MARKETING';
  UPDATE DEPART SET LOC = 'SEVILLA' WHERE DNOMBRE=nomDepto;
  IF SQL%NOTFOUND THEN
    DBMS_OUTPUT.PUT_LINE('Error en la actualización');
  END IF;
  DBMS_OUTPUT.PUT_LINE('Continúa la ejecución');
  SELECT LOC INTO locDepto FROM DEPART WHERE DNOMBRE=nomDepto;
  IF SQL%NOTFOUND THEN
    DBMS_OUTPUT.PUT_LINE('ESTO NO SE EJECUTA NUNCA');
  END IF;
END;
```

B.B.D.D.



Nuria Fuentes Pérez

5. CURSORES.

B. Cursores explícitos.

Operaciones básicas:

i. **Declaración.** Se declara en la zona de declaraciones

CURSOR <nombreCursor> **IS** <sentenciaSelect>;

ii. **Apertura.** Se ejecuta la sentencia SELECT

OPEN <nombreCursor>;

iii. **Recogida de información.**

FETCH <nombreCursor> **INTO** {<variable>|<listaVariables>;}

Cada sentencia FETCH recupera una fila del cursor y el puntero avanza automáticamente a la siguiente fila.

<variable> <nombreCursor>%ROWTYPE;

<listaVariables> del mismo tipo que las columnas del cursor.

iv. **Cierre.**

CLOSE <nombreCursor>;

B.B.D.D.



Nuria Fuentes Pérez

Recorrer un fichero secuencial.



- `ff(nfichero)`: devuelve verdadero si en la última operación de lectura se leyó la marca de fin de fichero.
- `ff(nfichero)`: devuelve verdadero si el puntero del fichero está al final de fichero.

```
Abrir nFichero para lectura
Leer nFichero,nRegistro
Mientras no ff(nFichero) hacer
    // tratamiento del registro
    Leer nFichero, nRegistro
Fin Mientras
Cerrar nFichero
```

```
Abrir nFichero para lectura
Mientras no ff(nFichero) hacer
    Leer nFichero, nRegistro
    // tratamiento del registro
Fin Mientras
Cerrar nFichero
```


5. CURSORES.

B. Cursores explícitos.

- **nCursor%FOUND.** → Booleano. Devuelve *true* si el último FETCH ha recuperado algún valor, en otro caso devuelve *false*. Después de abrir el cursor y antes de FETCH toma el valor *null*.
- **nCursor%NOTFOUND.** → Booleano. NOT nCursor%FOUND

```
DECLARE
CURSOR CUR1 IS
    SELECT DNOMBRE,LOC
    FROM DEPART;
nomDepto DEPART.DNOMBRE%TYPE;
locDepto DEPART.LOC%TYPE;
BEGIN
    OPEN cur1;
    FETCH cur1 INTO nomDepto,locDepto;
    WHILE cur1%FOUND LOOP
        DBMS_OUTPUT.PUT_LINE(nomDepto||' * '||locDepto);
        FETCH cur1 INTO nomDepto,locDepto;
    END LOOP;
    CLOSE cur1;
END;
```

```
OPEN cur1;
LOOP
    FETCH cur1 INTO nomDepto,locDepto;
    EXIT WHEN cur1%NOTFOUND;
    DBMS_OUTPUT.PUT_LINE(nomDepto||' * '||locDepto);
END LOOP;
CLOSE cur1;
```

B.B.D.D.



Nuria Fuentes Pérez

5. CURSORES.

B. Cursores explícitos.

- **nCursor%ROWCOUNT.** → número entero.
Devuelve el número de filas recuperadas hasta el momento por el cursor.
- **nCursor%ISOPEN.** → booleano. Devuelve *true* si el cursor está abierto, y *false* en caso contrario.

CURSOR CUR1 IS

SELECT DNOMBRE,LOC FROM DEPART WHERE DEPT_NO = departamento;

Cursor FOR...LOOP

```
DECLARE
CURSOR CUR1 IS
SELECT DNOMBRE,LOC FROM DEPART;
BEGIN
FOR reg IN cur1 LOOP
DBMS_OUTPUT.PUT_LINE(reg.dnombre||' * '||reg.loc);
END LOOP;
END;
```

Variable declarada implícitamente, local al bucle.

Acceso a los campos del registro.

B.B.D.D.



Nuria Fuentes Pérez

6. EXCEPCIONES.

```
EXCEPTION  
WHEN <nombreExcepcion> THEN  
    <bloqueInstrucciones>;  
WHEN <nombreExcepcion> THEN  
    <bloqueInstrucciones>;  
...  
[WHEN OTHERS THEN  
    <bloqueInstrucciones>;]
```

Las excepciones pueden ser:

- Internas predefinidas.
- Definidas por el usuario.
- Otras excepciones.

6. EXCEPCIONES.

A. Predefinidas.

ACCESS_INTO_NULL
COLLECTION_IS_NULL
CURSOR_ALREADY_OPEN
DUP_VAL_ON_INDEX
INVALID_CURSOR
INVALID_NUMBER
LOGIN_DENIED
NOT_LOGGED_ON
NO_DATA_FOUND
PROGRAM_ERROR
ROWTYPE_MISMATCH
SUBSCRIPT_OUTSIDE_LIMIT
STORAGE_ERROR
TIMEOUT_ON_RESOURCE
TOO_MANY_ROWS
VALUE_ERROR
ZERO_DIVIDE

...

```
EXCEPTION  
WHEN NO_DATA_FOUND THEN  
    DBMS_OUTPUT.PUT_LINE('datos no encontrados');  
WHEN TOO_MANY_ROWS THEN  
    DBMS_OUTPUT.PUT_LINE('demasiadas filas');  
END;
```

B.B.D.D.



Nuria Fuentes Pérez

6. EXCEPCIONES.

A. Predefinidas.

```
DECLARE
  nomDepto DEPART.DNOMBRE%TYPE;
  locDepto DEPART.LOC%TYPE;
BEGIN
  nomDepto:='MARKETING';
  UPDATE DEPART SET LOC = 'SEVILLA' WHERE DNOMBRE=nomDepto;
  IF SQL%NOTFOUND THEN
    DBMS_OUTPUT.PUT_LINE('Error en la actualización');
  END IF;
  DBMS_OUTPUT.PUT_LINE('Continúa la ejecución');
  SELECT LOC INTO locDepto FROM DEPART WHERE DNOMBRE=nomDepto;
EXCEPTION
  WHEN NO_DATA_FOUND THEN
    DBMS_OUTPUT.PUT_LINE('ERROR: no existe el departamento');
END;
```

6. EXCEPCIONES.

B. Definidas por el usuario.

1. Declarar la excepción.

```
DECLARE  
    <nombreExcepción> EXCEPTION;
```

2. Lanzar o disparar la excepción.

```
BEGIN  
    ...  
    RAISE <nombreExcepción>;  
    ...  
END;
```

3. Tratamiento de la excepción.

```
EXCEPTION  
    WHEN <nombreExcepción> THEN  
        <tratamiento>;
```



6. EXCEPCIONES.

C. Otras excepciones.

```
EXCEPTION
...
WHEN OTHERS THEN
    DMBS_OUTPUT.PUT_LINE('Error:'||SQLCODE||SQLERRM);
...
END;
```

ASOCIAR UNA EXCEPCIÓN A ALGUNO DE ESTOS ERRORES INTERNOS

1. Declarar la excepción.
 <nombreExcepción> **EXCEPTION**;
2. Asociar la excepción al error.
 PRAGMA EXCEPTION_INIT(<nombreExcepción>,<numErrorOracle>);
2. Tratamiento de la excepción.
 EXCEPTION
 WHEN <nombreExcepción> **THEN**
 <tratamiento>;



6. EXCEPCIONES.

Propagación de excepciones.

Bloque 1
BEGIN

...
Llamada a bloque 2

....
EXCEPTION
...
END;

Bloque 2



BEGIN

...



EXCEPTION

...



END;

RAISE_APPLICATION_ERROR(numError,mensajeError);

Cuando un programa hace esta llamada, se lanza la excepción y se deshacen los cambios realizados por el programa.

7. Cursores de actualización

Cursores que se usan para actualizar o borrar filas del mismo.

CURSOR <nombreCursor> **IS** <sentenciaSelect> **FOR UPDATE**;

Las filas seleccionadas serán bloqueadas tan pronto se abra el cursor y serán desbloqueadas al terminar las actualizaciones, implícita o explícitamente COMMIT.

Para actualizar:

UPDATE ... WHERE CURRENT OF nCursor;

Para borrar:

DELETE ... WHERE CURRENT OF nCursor;

Si el cursor hace referencia a varias tablas, se deberá usar:

CURSOR ... FOR UPDATE OF <nombreColumna>;

Con lo que únicamente se bloquearán las filas correspondientes de la tabla que tenga la columna especificada.

7. Cursores de actualización

```
CREATE OR REPLACE PROCEDURE subirSalarioDepto(numDepto NUMBER,  
                                              subida NUMBER)  
IS  
  CURSOR curEmple IS SELECT OFICIO,SALARIO FROM EMPL  
                      WHERE DEPT_NO=numDepto FOR UPDATE;  
  regCurEmple curEmple%ROWTYPE;  
  inc NUMBER(8,2);  
BEGIN  
  OPEN curEmple;  
  FETCH curEmple INTO regCurEmple;  
  WHILE curEmple%FOUND LOOP  
    inc:=(regCurEmple.salario/100)*subida;  
    UPDATE EMPL SET SALARIO=SALARIO+inc  
              WHERE CURRENT OF curEmple;  
    FETCH curEmple INTO regCurEmple;  
  END LOOP;  
  CLOSE curEmple;  
END subirSalarioDepto;
```

B.B.D.D.



Nuria Fuentes Pérez

7. Cursores de actualización

Con un cursor FOR UPDATE:

- se bloquean **todas** las filas del SELECT cuando se abre el cursor.
- si se ejecuta un COMMIT, después ya no se puede ejecutar FETCH.

Un forma de evitarlo:

```
CURSOR nCursor IS SELECT <col1>,<col2>,...,ROWID FROM <tabla>;
```

```
UPDATE ... WHERE ROWID=nCursor.rowid;
```

```
DELETE ... WHERE ROWID=nCursor.rowid;
```

7. Cursores de actualización

```
CREATE OR REPLACE PROCEDURE subirSalarioDepto(numDepto NUMBER,  
                                              subida NUMBER)  
IS  
  CURSOR curEmple IS SELECT OFICIO,SALARIO, ROWID FROM EMPL  
                      WHERE DEPT_NO=numDepto;  
  regCurEmple curEmple%ROWTYPE;  
  inc NUMBER(8,2);  
BEGIN  
  OPEN curEmple;  
  FETCH curEmple INTO regCurEmple;  
  WHILE curEmple%FOUND LOOP  
    inc:=(regCurEmple.salario/100)*subida;  
    UPDATE EMPL SET SALARIO=SALARIO+inc  
                WHERE ROWID = regCurEmple.rowid;  
    FETCH curEmple INTO regCurEmple;  
  END LOOP;  
END subirSalarioDepto;
```

B.B.D.D.



Nuria Fuentes Pérez

8. TRIGGERS

Un disparador de base de datos es un programa PL/SQL asociado con una **tabla** de la base de datos.

Oracle ejecuta el disparador cada vez que una operación de SQL, de las definidas en el trigger, afecta a la tabla.

Se pueden utilizar disparadores de base de datos para:

- ✓ Controlar la *modificación* de las tablas.
- ✓ Forzar *reglas* de validación complejas.
- ✓ *Calcular* valores de columnas (derivados de otras columnas) de manera automática.
- ✓ Realizar controles de *seguridad* complejos.
- ✓ Mantener tablas *replicadas*.
- ✓ Crear tablas de *datos históricos*
- ✓ Forzar actualizaciones y borrados *en cascada*
- ✓ Etc....

8. TRIGGERS

- ❑ **NOMBRE DEL TRIGGER.**
- ❑ **EVENTO DE DISPARO.** Se indica:
 - La tabla a la que está asociada.
 - El evento que producirá que el trigger sea "*disparado*".
(INSERT,DELETE, UPDATE column)
- ❑ **TIPO DEL TRIGGER.** Indica:
 - El momento en que se ejecuta el trigger (BEFORE, AFTER).
 - El nivel de disparo del trigger, que puede ser:
 - A nivel de orden (por defecto). FOR EACH STATEMENT.
 - A nivel de fila. FOR EACH ROW.
- ❑ **RESTRICCIÓN DEL TRIGGER.** Restringe la ejecución del trigger al cumplimiento de la condición especificada (WHEN). Limitaciones:
 - Solo se puede usar con triggers a nivel de fila.
 - Se trata de una condición SQL (no PL/SQL)
 - No puede incluir una consulta a la misma o a otras tablas o vistas.
- ❑ **CUERPO DEL TRIGGER.** Código a ejecutar.

8. TRIGGERS BEFORE / AFTER

```
CREATE [OR REPLACE] TRIGGER nTrigger
{BEFORE | AFTER} {DELETE|INSERT|UPDATE [OF <lista_columnas>]}
[OR {BEFORE | AFTER} {DELETE|INSERT|UPDATE [OF <lista_columnas>]}]
ON {nTabla | nVista}
[REFERENCING {NEW | OLD} AS nuevo [, {NEW | OLD} AS nuevo]]
[FOR EACH {STATEMENT|ROW [WHEN (condicion)]}]
[DECLARE
  <declaraciones>]
BEGIN
  <sentencias>
[EXCEPTION
  <gestión de excepciones>]
END;
```

En Trigger a nivel de fila, al actualizar una fila...

:old.nColumna
:new.nColumna

Con DELETE → :new.nColumna (es null)
Con INSERT → :old.nColumna (es null)
Con UPDATE → tienen sentido ambos

8. TRIGGERS. EJEMPLO

```
CREATE OR REPLACE TRIGGER auditaBorradoEmple  
BEFORE DELETE  
ON EMPL  
FOR EACH ROW  
BEGIN  
    INSERT INTO AUDITAEMPLE  
        VALUES ('BORRADO EMPLEADO*' || :old.EMP_NO || '*'  
                || :old.APELLIDO || '*' || :old.DEPT_NO);  
END;
```


8. TRIGGERS BEFORE / AFTER

Para controlar varios eventos sobre una misma tabla o vista:

A. Crear un trigger para cada evento. El orden de ejecución de éstos será:

I. Antes de ejecutar la orden que produce el disparo se ejecutarán los triggers BEFORE... FOR EACH STATEMENT.

II. Para cada fila afectada por la orden:

1. Triggers BEFORE... FOR EACH ROW

2. Se ejecuta la actualización de la fila y se bloquea hasta que se confirme la transacción.

3. Triggers AFTER... FOR EACH ROW

III. Una vez realizada la actualización se ejecutarán los trigger AFTER ... FOR EACH STATEMENT.

B. Un trigger que controle distintos eventos BEFORE DELETE OR UPDATE ON columna ...

IF **INSERTING** ...

IF **DELETING** ...

IF **UPDATING**...

IF **UPDATING**('nColumna')...

B.B.D.D.



Nuria Fuentes Pérez

8. TRIGGERS BEFORE / AFTER

Puede contener consultas, manipulación de datos y llamadas a otros subprogramas. **No puede:**

- Contener sentencias de control de transacciones (COMMIT, ROLLBACK)
- Llamadas a procedimientos con sentencias de control de transacciones.
- Comandos DDL.
- Instrucciones que consulten o modifiquen tablas mutantes.
- Cambiar valores de las columnas que sean claves primarias, únicas o ajenas.

Los trigger FOR EACH STATEMENT no se verán afectados por estas restricciones, excepto cuando el trigger se dispare como resultado de una restricción ON DELETE CASCADE.

8. TRIGGERS SUSTITUCIÓN

No se ejecutan ni antes ni después, sino en lugar de la orden de actualización.

```
CREATE [OR REPLACE] TRIGGER nTrigger  
INSTEAD OF {DELETE|INSERT|UPDATE}  
  [OR {DELETE|INSERT|UPDATE}]  
ON nVista  
FOR EACH ROW  
[DECLARE  
  <declaraciones>  
BEGIN  
  <sentencias>  
[EXCEPTION  
  <gestión de excepciones>  
END;
```

- Solo se pueden usar con vistas
- Actúan siempre a nivel de fila, por defecto, **FOR EACH ROW**
- No se puede especificar una restricción de disparo (**WHEN**)

8. TRIGGERS SUSTITUCIÓN

Sea la siguiente vista:

```
CREATE VIEW VISTA_EMPLE AS  
SELECT EMP_NO, APELLIDO, OFICIO,  
DNOMBRE, LOC FROM EMPL, DEPART WHERE  
EMPL.DEPT_NO=DEPART.DEPT_NO;
```

Hay operaciones como eliminar un empleado,
`DELETE FROM VISTA_EMPLE WHERE EMP_NO=7839;`

insertar una dato nuevo en la vista

```
INSERT INTO VISTA_EMPLE VALUES  
(7999,'FUENTES','VENDEDOR','CONTABILIDAD','SEVILLA');
```

o actualizar el nombre del departamento de un empleado de la vista

```
UPDATE VISTA_EMPLE SET DNOMBRE='CONTABILIDAD' WHERE APELLIDO='SALA';
```

que no se pueden realizar directamente.

Hagamos un trigger sobre la vista que permita realizar cualquiera de estas operaciones.

```

CREATE OR REPLACE TRIGGER gestionVistaEmple
  INSTEAD OF INSERT OR UPDATE ON VISTA_EMPL
  FOR EACH ROW
  DECLARE
    vDept DEPART.DEPT_NO%TYPE;
    vCambio NUMBER(1):=0;
  BEGIN
    IF INSERTING THEN
      SELECT DEPT_NO INTO vDept FROM DEPART WHERE DEPART.DNOMBRE=:new.DNOMBRE AND LOC = :new.LOC;
      INSERT INTO EMPL (EMP_NO,APELLIDO,OFICIO,DEPT_NO) VALUES (:new.EMP_NO,:new.APELLIDO,:new.OFICIO,vDept);
    ELSE
      IF UPDATING('EMP_NO') OR UPDATING('APELLIDO') OR UPDATING('OFICIO') THEN
        UPDATE EMPL SET EMP_NO=:new.EMP_NO, APELLIDO=:new.APELLIDO,OFICIO=:new.OFICIO
          WHERE EMP_NO = :old.EMP_NO;
      ELSE
        vCambio:=1;
        IF UPDATING('DNOMBRE') THEN
          SELECT DEPT_NO INTO vDept FROM DEPART WHERE DEPART.DNOMBRE=:new.DNOMBRE;
          UPDATE EMPL SET DEPT_NO = vDept WHERE EMP_NO = :old.EMP_NO;
        END IF;
      END IF;
    END IF;
  EXCEPTION
    WHEN NO_DATA_FOUND THEN
      IF vCambio = 0 THEN
        DBMS_OUTPUT.PUT_LINE('No existe el departamento a añadir '||:new.DNOMBRE);
      ELSE
        DBMS_OUTPUT.PUT_LINE('No existe el departamento a MODIFICAR '||:new.DNOMBRE);
      END IF;
  END;

```

B.B.D.D.



Nuria Fuentes Pérez

8. TRIGGERS SUSTITUCIÓN

- ❑ Privilegios:
 - **CREATE TRIGGER** → crear trigger en el esquema actual
 - **CREATE ANY TRIGGER** → en cualquier esquema
 - Privilegios sobre las tablas o vistas, los del propietario del trigger son los privilegios con los que trabaja el trigger cuando se dispara.
 - El trigger forma parte de la operación de actualización que lo disparó, si este falla, la actualización no se lleva a cabo. (Levantar una excepción y no tratarla)
- ❑ Activar trigger:
 - **ALTER TRIGGER** nTrigger **ENABLE**;
- ❑ Desactivar el trigger:
 - **ALTER TRIGGER** nTrigger **DISABLE**;
- ❑ Para recompilar un trigger:
 - **ALTER TRIGGER** nTrigger **COMPILE**;
- ❑ Eliminar un trigger:
 - **DROP TRIGGER** nTrigger;

VISTAS
DBA_TRIGGERS
USER_TRIGGERS

9. REGISTROS

- ❑ Estructura compuesta de otras más simples, llamadas campos, que pueden ser de distintos tipos.
- ❑ ROWTYPE: es un registro aunque con limitaciones
 - Los nombres de columnas y las tipos quedan limitados a los de la tabla, sin posibilidad de excluir campos o incluir otros.
 - No se incluyen restricciones NOT NULL o valores por defecto.

- ❑ Se define el tipo:

```
TYPE <nombreTipo> IS RECORD(  
    <nombreCampo1> tipo [[NOT NULL]]{:=|DEFAULT} <valorInicial>],  
    <nombreCampo2> tipo [[NOT NULL]]{:=|DEFAULT} <valorInicial>],  
    ...  
);
```

Si se especifica NOT NULL, el campo debe ser inicializado.

- ❑ Se define la variable:

```
<nombreVariable> <nombre_tipo>;
```

- ❑ reg1:=reg2; ➔ Variables definidas sobre el mismo tipo registro.

- ❑ <nombreRegistro>.<nombreCampo>

10. TABLAS

- ☐ Define estructuras similares a los arrays de otros lenguajes.
- ☐ Estáticos. Tamaño fijo.
- ☐ Se define el tipo:

TYPE nombreTipo **IS** **VARRAY**(numeroElementos) **OF** tipoElementos ;

- ☐ Se define la variable:

<nombreTabla> <nombreTipo>;

- ☐ Cargar por defecto

<nombreTabla>:= <nombreTipo>(<valor>[,<valor>...]);

- ☐ nombreTabla(numeroElemento)

10. TABLAS

- ☐ Define estructuras similares a los arrays de otros lenguajes.
- ☐ Tienen dos columnas:
 - Una es el índice o clave de tipo `BINARY_INTEGER`
 - La otra guarda la información definida por el programador.
- Características:
 - Los índices no tienen por qué ser secuencial.
 - Los elementos no tienen por qué ocupar posiciones contiguas de memoria.
 - No tienen una longitud predeterminada, el nº de elementos puede incrementarse dinámicamente.

- ☐ Se define el tipo:

TYPE nombreTipo ***IS TABLE OF*** tipoElementos [***NOT NULL***]
INDEX BY BINARY_INTEGER

- ☐ Se define la variable:

<nombreTabla> <nombreTipo>;

- ☐ nombre_tabla(numeroElemento)

B.B.D.D.



Nuria Fuentes Pérez

10. TABLAS

□ `<variableTabla>.FIRST` || `<variableTabla>.LAST` || `<variableTabla>.PRIOR(n)` ||
`<variableTabla>.NEXT(n)` || `<variableTabla>.COUNT` ||
`<variableTabla>.EXISTS(n)` || `<variableTabla>.DELETE[(n1[n2])]`